

Ejercicios

Ejercicio 2 — Explorar el historial

- Se han registrado 3 versiones del modelo en el archivo `training_history.json`: `v1.0-base`, `v2.0-24067e` y `v3.0-70b3d2`.
- Activadas: `v1.0-base` y `v2.0-24067e`. Rechazadas: `v3.0-70b3d2`
- El modelo `v3.0-70b3d2` fue rechazado porque obtuvo una `accuracy` de 0.6, que es inferior a la del modelo activo (1.0). El sistema de `continuous training` solo activa un nuevo modelo si su rendimiento es igual o mejor que el modelo actual. Como el modelo entrenado con muestras ruidosas (etiquetas incorrectas) empeoró el rendimiento, no se activó y quedó marcado como `"status": "rechazado"`.

Ejercicio 3 — Entrenamiento incremental vs. desde cero

En el primer experimento se realizó una llamada a `/train` con `retrain_from_scratch = false`. En este caso, el entrenamiento fue incremental, es decir, el modelo utilizó las nuevas muestras junto con los datos previamente acumulados. El resultado fue un modelo activado con una `accuracy` de 0.5333, que mejora respecto al modelo anterior (0.4000), por lo que el sistema activó la nueva versión siguiendo la regla del `quality gate` (`accuracy_new >= accuracy_previous`).

En el segundo experimento se intentó entrenar con `retrain_from_scratch = true`, lo que implica entrenar el modelo solo con las muestras enviadas en la petición, sin utilizar datos previos. Sin embargo, el sistema devolvió un error indicando que el dataset no contenía al menos dos clases distintas, condición necesaria para poder entrenar un modelo de clasificación. Por esta razón, el entrenamiento no se ejecutó.

Esto demuestra que el entrenamiento incremental es más robusto cuando el conjunto de nuevas muestras es pequeño o incompleto, ya que puede aprovechar los datos acumulados previamente para entrenar el modelo.

Preguntas

Pregunta 1

No necesariamente se obtendría el mismo modelo ni el mismo `accuracy`. Aunque el dataset de Iris sea el mismo, durante el proceso de entrenamiento suele existir algún componente aleatorio, como la división entre datos de entrenamiento y validación. Si en el código se fija una semilla de aleatoriedad mediante un parámetro como `random_state`, el resultado será reproducible y se obtendrá siempre el mismo modelo y `accuracy`. Si no se fija ese parámetro, cada ejecución puede generar una división distinta de los datos y, por tanto, resultados ligeramente diferentes.

Pregunta 2

Si se enviaran 10 muestras todas de la clase setosa, el modelo no se entrenaría. El endpoint /train comprueba que el dataset tenga al menos dos clases distintas antes de iniciar el entrenamiento. Al detectar que todas las muestras pertenecen a la misma clase, la API devolvería un error indicando que el dataset debe contener al menos dos clases.

Pregunta 3

El sistema guarda también los modelos rechazados en el historial para mantener un registro completo de todos los intentos de entrenamiento. Esto permite auditar qué datos se utilizaron y cómo afectaron al rendimiento del modelo. Si solo se guardaran los modelos activados, se perdería información importante sobre entrenamientos que empeoraron el rendimiento. Conservar estos registros ayuda a entender por qué un modelo fue rechazado y facilita detectar problemas como datos ruidosos o mal etiquetados.

Pregunta 4

Usar la condición $\text{accuracy_nuevo} \geq \text{accuracy_anterior}$ puede ser razonable porque, cuando el modelo ya está cerca de su rendimiento máximo, el accuracy puede dejar de mejorar aunque el dataset siga creciendo con datos de buena calidad. En ese caso, un modelo nuevo con el mismo accuracy podría estar entrenado con más datos y ser más estable o generalizar mejor. Si se utilizara un $>$ estricto, muchos modelos potencialmente válidos no se activarían simplemente porque la métrica no mejora ligeramente, aunque el modelo sea igual de bueno o incluso más robusto.